SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1003	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	ALIGETI AKSHAYA	**Reg. No.:**	192425170
Date:	04-08-2026	**Duration:**	60 minutes

Code of Conduct

I _A.Akshaya(192425170)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Akshaya_____

Annexure A – Architecture Design Assignment

ACADEMIC PROJECT, ALLOCATION MONITORING AND EVALUATION MANAGEMENT SYSTEM

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

  The **Academic Project Allocation, Monitoring and Evaluation Management System** is developed to improve the way academic projects are assigned, monitored, and evaluated in colleges. In the existing system, projects are mostly allocated based on students' CGPA without considering their interests, technical skills, or preferred domains. This may reduce students' motivation and learning opportunities. The proposed system aims to allocate projects based on students' interests and abilities so that they can work in their preferred domain. It also helps faculty members monitor project progress, evaluate students fairly, and maintain project records digitally. The system reduces manual work, improves transparency, and enhances communication between students, faculty, HOD, project coordinators, and management.

  The main objectives are:

  - Allocate projects based on students' interests and technical skills.
  - Improve practical learning and project quality.
  - Monitor project progress efficiently.
  - Maintain project records digitally.
  - Make project evaluation fair and transparent.
  - Improve communication among all stakeholders.
  - Reduce manual work and save time.

- Analyze the functional and non-functional requirements that influence the software architecture.

Functional Requirements

- ❖ User Registration and Login.
- ❖ Student Profile Management.
- ❖ Interest and Skill Selection.
- ❖ Project Allocation.
- ❖ Faculty Guide Assignment.
- ❖ Project Progress Tracking.
- ❖ Internal and External Evaluation.
- ❖ Feedback Management.

- ❖ Report Generation.
- ❖ User and Project Management.

Non-Functional Requirements

- ❖ Fast system performance.
- ❖ Secure login and data protection.
- ❖ High reliability with minimum failures.
- ❖ Easy-to-use interface.
- ❖ Ability to support more users in the future.
- ❖ Easy maintenance and software updates.
- ❖ High system availability.
- ❖ Regular backup and data recovery.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.
    - ✓ **Scalability:** The system should support an increasing number of students, faculty members, departments, and projects without reducing performance.
    - ✓ **Maintainability:** The software should be easy to modify, update, and maintain whenever new features or changes are required.
    - ✓ **Reliability:** The system should work correctly with minimal errors and should securely store all project-related data.
    - ✓ **Availability:** The application should be available whenever students and faculty need to access it, especially during project submission and evaluation periods.
    - ✓ **Performance:** The system should respond quickly during login, project allocation, report generation, and database operations.
    - ✓ **Security:** The system should provide secure login, role-based access control, password protection, encrypted data storage, and ensure that only authorized users can access sensitive information.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

  For the **Academic Project Allocation, Monitoring and Evaluation Management System**, the most suitable architecture is the **Layered Architecture (Three-Tier Architecture)**.

  The system is divided into three layers:

- **Presentation Layer (Frontend):** Provides the user interface where students, faculty, HOD, project coordinators, administrators, and external examiners interact with the system.
- **Business Logic Layer (Application Layer):** Processes user requests, applies business rules, allocates projects, manages evaluations, and handles all system operations.
- **Data Layer (Database Layer):** Stores student details, faculty information, project data, progress reports, evaluation marks, feedback, and other records securely.

  This architecture separates the user interface, business logic, and database, making the system well-organized and easier to manage.

- Justify why the selected architecture is the most suitable for the given problem.

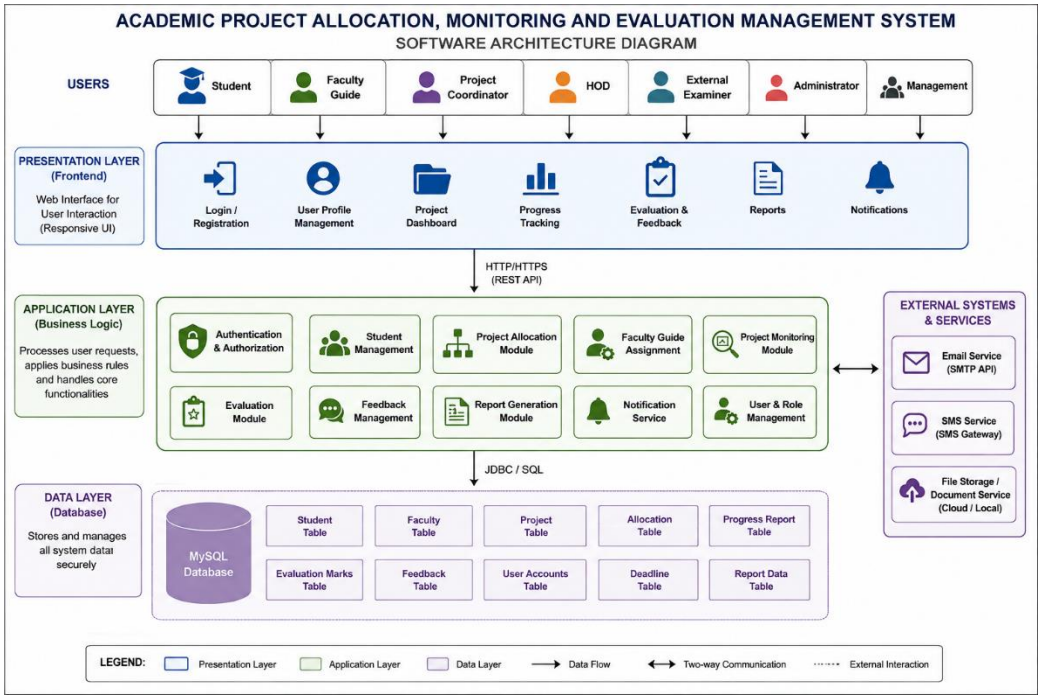
  The **Layered Architecture** is the best choice because it provides a clear separation of responsibilities, making the application easier to develop, test, and maintain. Since this system has multiple users such as students, faculty guides, HODs, project coordinators, administrators, and external examiners, the layered approach helps each module function independently while sharing the same database.

  Reasons for choosing Layered Architecture:

- It separates the frontend, business logic, and database into different layers.
- It is easy to develop, understand, and maintain.
- Changes made in one layer do not significantly affect the other layers.
- It improves code reusability and reduces complexity.
- It supports secure communication between users and the database.
- It allows easy integration of new features in the future.
- It is suitable for web-based full-stack applications.
- It provides better scalability as the number of users increases.
- It improves system reliability and performance.
- It makes testing and debugging easier because each layer can be tested separately.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.



- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

Major Components

Presentation Layer

- Login & Registration
- Student Dashboard
- Faculty Dashboard
- HOD Dashboard
- Admin Dashboard
- Project Coordinator Dashboard

Business Logic Layer

- Authentication
- Student Profile Management
- Project Allocation
- Faculty Assignment
- Project Monitoring
- Evaluation
- Report Generation
- Notification

Database Layer

- Student Details
- Faculty Details
- Project Details
- Allocation Records
- Progress Reports
- Evaluation Marks
- Feedback
- User Accounts

External Systems

- Email/SMS Notification API
- File Storage for Project Documents

Communication Mechanism

- **HTTP/HTTPS** → Client to Frontend
- **REST API** → Frontend to Business Logic
- **SQL** → Business Logic to Database
- **SMTP/API** → Notifications

- Use appropriate software architecture notations and labels.
    - **Rectangles** → Layers/Components
    - **Database Cylinder (optional while drawing)** → MySQL Database
    - **Arrows** → Data Flow/Communication
    - **External Boxes** → External APIs/Services

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

The Academic Project Allocation, Monitoring and Evaluation Management System follows a Layered Architecture**,** where each layer has a specific responsibility.

Presentation Layer (Frontend)

The Presentation Layer is the interface through which users interact with the system. It provides login pages, dashboards, project details, report submission forms, and evaluation screens. It collects user input and displays the required information in a simple and user-friendly manner.

Business Logic Layer (Application Layer)

The Business Logic Layer processes all user requests and performs the core functions of the system. It verifies login credentials, manages student profiles, allocates projects based on interests and skills, monitors project progress, evaluates projects, generates reports, and sends notifications. This layer acts as the bridge between the frontend and the database.

Data Layer (Database)

The Data Layer stores all the information related to students, faculty, projects, project allocations, progress reports, evaluation marks, feedback, deadlines, and user accounts. It ensures that data is stored securely and can be retrieved whenever required.

External Systems

External services such as Email API, SMS Gateway, and File Storage are used to send notifications, store project documents, and improve communication between users.

- Explain how the components communicate and exchange data.

    1. The user accesses the system through the **Presentation Layer** using a web browser.
    2. The Presentation Layer sends the user's request to the **Business Logic Layer** through **HTTP/HTTPS (REST API)**.
    3. The Business Logic Layer processes the request by applying business rules and validations.
    4. If data is required, it sends **SQL queries** to the **Database Layer**.
    5. The Database returns the requested information to the Business Logic Layer.
    6. The processed result is sent back to the Presentation Layer.
    7. The Presentation Layer displays the result to the user.
    8. If required, the Business Logic Layer also communicates with **Email or SMS services** to send notifications about project allocation, deadlines, evaluations, or feedback.

- Illustrate the overall workflow of the system.

    The overall workflow of the system begins when a student registers and logs into the application. The student creates a profile by entering personal details, selecting areas of interest, preferred domains, and technical skills. Based on this information, the Project Coordinator allocates a suitable project and assigns a faculty guide. The student can then view the allocated project and upload regular progress reports. Faculty members review the submitted work, provide guidance, and monitor the student's progress throughout the project. At the end of the project, both the faculty guide and the external examiner evaluate the project and submit the marks. The HOD and college management can monitor project status and generate reports whenever required. All project records, feedback, evaluations, and reports are securely stored in the database, making the entire project management process transparent, efficient, and easy to maintain.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability

  The system is designed to support a growing number of students, faculty members, departments, and projects. Since the presentation layer, business logic layer, and database layer are separated, new users and features can be added without affecting the existing system. As the institution grows, the architecture can easily handle increased workloads.

- Security

  The architecture provides strong security by using secure login, password authentication, and role-based access control. Different users such as students, faculty, HOD, project coordinators, administrators, and external examiners can access only the information related to their roles. Sensitive information such as student records, project details, and evaluation marks is stored securely in the database to prevent unauthorized access.

- Performance

  The layered architecture improves system performance by processing requests efficiently. The presentation layer handles user interactions, while the business logic layer performs calculations and validations before accessing the database. This reduces unnecessary database operations and allows users to receive faster responses during login, project allocation, report generation, and project monitoring.

- Reliability

  The system is designed to work continuously with minimum errors and failures. All project information, student records, evaluation marks, and reports are stored accurately in the database. Regular data validation and backup mechanisms help prevent data loss and ensure that users receive reliable information whenever they access the system.

- Maintainability

  The separation of the system into different layers makes it easy to maintain and update. If any changes are required, such as adding new modules or improving existing features, developers can modify one layer without affecting the others. This reduces development time, simplifies debugging, and improves software quality.

- Availability

  The system is designed to be available whenever users need it. Students, faculty members, and administrators can access the application anytime through an internet connection. Proper server management, regular backups, and system monitoring help reduce downtime and ensure that the application remains accessible during project submission, evaluation, and report generation.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

  The **Layered Architecture (Three-Tier Architecture)** was selected because it provides a clear separation between the user interface, business logic, and database. This structure makes the system easy to understand, develop, and manage. Since the **Academic Project Allocation, Monitoring and Evaluation Management System** has multiple users such as students, faculty, HOD, project coordinators, administrators, and external examiners, the layered architecture allows each module to perform its specific function independently while working together as one complete system. It also improves code organization, reduces complexity, and ensures secure communication between different components.

- Discuss the trade-offs considered while selecting the architecture.

  Advantages

- Easy to develop and understand.
- Clear separation of frontend, business logic, and database.
- Easier testing, debugging, and maintenance.
- Better security through role-based access.
- Suitable for full-stack web applications.
- Easy to add new features in the future.

  Trade-offs

- Communication between multiple layers may slightly increase response time.
- More files and modules are required compared to a simple application.
- Initial development may take more time because each layer is developed separately.
- Large systems may require additional optimization to maintain high performance.

- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

  The Layered Architecture makes the system flexible and easy to improve in the future. New features such as **AI-based project recommendations**, **online viva management**, **plagiarism checking**, **email and SMS notifications**, **cloud storage**, and **mobile application support** can be added without changing the entire system. The architecture also supports integration with external systems like Learning Management Systems (LMS), university portals, email services, cloud databases, and third-party APIs. Since each layer works independently, developers can update or replace one layer without affecting the others. This improves long-term maintainability, reduces development effort, and allows the system to grow as the number of students, faculty members, and projects increases. Overall, the chosen architecture provides a scalable, secure, maintainable, and future-ready solution for managing academic projects efficiently.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

	organized, professional report.			

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25